server this is often combined with a timestamp, so that used nonces only have to be remembered until the associated request expires. In some cases, you may be able to use a *monotonically increasing counter* as the nonce, in which case the server only needs to remember the highest value it has seen so far and reject requests that use a smaller value. If multiple clients or servers share the same key it can be difficult to synchronize the counter between them all.

- The most secure method is to use a *challenge-response protocol* shown in figure 13.3, in which the server generates a random challenge value (a nonce) and sends it to the client. The client then includes the challenge value in the API request, proving that the request was generated after the challenge. Although more secure, this adds overhead because the client must talk to the server to obtain a challenge before they can send any requests.

DEFINITION A ***monotonically increasing counter*** is one that only ever increases and never goes backwards and can be used as a nonce to prevent replay of API requests. In a ***challenge-response protocol*** the server generates a random challenge that the client includes in a subsequent request to ensure freshness.